

Individual Assignment 1

Individual Assignment 1

Class: DATA 611 Predictive Analysis

Semester: Winter 2026

Date: February 6, 2026

Student Number: 30139765

Student Name: Bhuvan Dubey

Individual Assignment 1

1. Classification Accuracy

a)

```
# 1. Classification Accuracy -----
# Data setup
data = {
    'propensity': [0.03, 0.52, 0.38, 0.82, 0.33, 0.42, 0.55, 0.59, 0.09, 0.21, 0.43, 0.04, 0.08, 0.13, 0.01, 0.79, 0.42, 0.29, 0.08, 0.02],
    'actual':      [0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
}
df = pd.DataFrame(data)

# 1a) Confusion matrix for cutoff values of 0.25 and 0.75
# True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN)
def compute_confusion_matrix(df, cutoff):
    df['predicted'] = (df['propensity'] >= cutoff).astype(int)
    TP = len(df[(df['actual'] == 1) & (df['predicted'] == 1)])
    TN = len(df[(df['actual'] == 0) & (df['predicted'] == 0)])
    FP = len(df[(df['actual'] == 0) & (df['predicted'] == 1)])
    FN = len(df[(df['actual'] == 1) & (df['predicted'] == 0)])
    return TP, TN, FP, FN

# Cutoffs
cutoffs = [0.25, 0.75]
for cutoff in cutoffs:
    TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
    print(f"\n--- Confusion matrix at cutoff {cutoff} ---")
    print(f"TP: {TP}")
    print(f"TN: {TN}")
    print(f"FP: {FP}")
    print(f"FN: {FN}")
```

```
>>> data = {
...     'propensity': [0.03, 0.52, 0.38, 0.82, 0.33, 0.42, 0.55, 0.59, 0.09, 0.21, 0.43, 0.04, 0.08, 0.13, 0.01, 0.79, 0.42, 0.29, 0.08, 0.02],
...     'actual':      [0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]
... }
>>>
>>> df = pd.DataFrame(data)
>>> # 1a) Confusion matrix for cutoff values of 0.25 and 0.75
>>> # True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN)
>>>
>>> def compute_confusion_matrix(df, cutoff):
...     df['predicted'] = (df['propensity'] >= cutoff).astype(int)
...     TP = len(df[(df['actual'] == 1) & (df['predicted'] == 1)])
...     TN = len(df[(df['actual'] == 0) & (df['predicted'] == 0)])
...     FP = len(df[(df['actual'] == 0) & (df['predicted'] == 1)])
...     FN = len(df[(df['actual'] == 1) & (df['predicted'] == 0)])
...     return TP, TN, FP, FN
...
... # Cutoffs
...
>>> cutoffs = [0.25, 0.75]
>>> for cutoff in cutoffs:
...     TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
...     print(f"\n--- Confusion matrix at cutoff {cutoff} ---")
...     print(f"TP: {TP}")
...     print(f"TN: {TN}")
...     print(f"FP: {FP}")
...     print(f"FN: {FN}")
...
--- Confusion matrix at cutoff 0.25 ---
TP: 3
TN: 9
FP: 8
FN: 0

--- Confusion matrix at cutoff 0.75 ---
TP: 2
TN: 17
FP: 0
FN: 1
```

Confusion matrix at cutoff 0.25

	Actual Positive Value	Actual Negative Value
Predicted Positive Value	3	8
Predicted Negative Value	0	9

Individual Assignment 1

Confusion matrix at cutoff 0.75

	Actual Positive Value	Actual Negative Value
Predicted Positive Value	2	0
Predicted Negative Value	1	17

1b)

```
# 1b) Calculate the PCC both error rates, sensitivity and specificity using cutoff values 0.25 and 0.75
def calculate_metrics(TP, TN, FP, FN):
    total = TP + TN + FP + FN
    PCC = (TP + TN) / total if total > 0 else 0
    error_rate = (FP + FN) / total if total > 0 else 0
    sensitivity = TP / (TP + FN) if (TP + FN) > 0 else 0
    specificity = TN / (TN + FP) if (TN + FP) > 0 else 0
    return PCC, error_rate, sensitivity, specificity

for cutoff in cutoffs:
    TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
    PCC, error_rate, sensitivity, specificity = calculate_metrics(TP, TN, FP, FN)
    print(f"\n--- Metrics at cutoff {cutoff} ---")
    print(f"PCC (Accuracy): {PCC:.2f}")
    print(f"Error Rate: {error_rate:.2f}")
    print(f"Sensitivity (Recall): {sensitivity:.2f}")
    print(f"Specificity: {specificity:.2f}")
```

```
>>> def calculate_metrics(TP, TN, FP, FN):
...     total = TP + TN + FP + FN
...     PCC = (TP + TN) / total if total > 0 else 0
...     error_rate = (FP + FN) / total if total > 0 else 0
...     sensitivity = TP / (TP + FN) if (TP + FN) > 0 else 0
...     specificity = TN / (TN + FP) if (TN + FP) > 0 else 0
...     return PCC, error_rate, sensitivity, specificity
...
>>> for cutoff in cutoffs:
...     TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
...     PCC, error_rate, sensitivity, specificity = calculate_metrics(TP, TN, FP, FN)
...     print(f"\n--- Metrics at cutoff {cutoff} ---")
...     print(f"PCC (Accuracy): {PCC:.2f}")
...     print(f"Error Rate: {error_rate:.2f}")
...     print(f"Sensitivity (Recall): {sensitivity:.2f}")
...     print(f"Specificity: {specificity:.2f}")
...

--- Metrics at cutoff 0.25 ---
PCC (Accuracy): 0.60
Error Rate: 0.40
Sensitivity (Recall): 1.00
Specificity: 0.53

--- Metrics at cutoff 0.75 ---
PCC (Accuracy): 0.95
Error Rate: 0.05
Sensitivity (Recall): 0.67
Specificity: 1.00
```

Individual Assignment 1

Metrics at cutoff 0.25

- PCC: 0.60
- Error Rate: 0.40
- Sensitivity: 1.00
- Specificity: 0.53

Metrics at cutoff 0.75

- PCC: 0.95
- Error Rate: 0.05
- Sensitivity: 0.67
- Specificity: 1.00

1c)

```
# 1c) Calculate the expected value for each cut-off value 0.25 and 0.75
cost_production = 15
sale_price = 25
total_samples = len(df)
actual_good = len(df[df['actual'] == 0])
actual_faulty = len(df[df['actual'] == 1])

for cutoff in cutoffs:
    TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
    false_negative_rate = FN / actual_faulty if actual_faulty > 0 else 0 # False negatives (faulty pieces missed)
    profit_good = (actual_good / total_samples) * (sale_price - cost_production) # Profit from correctly selling good pieces
    faulty_missed = false_negative_rate * actual_faulty
    profit_faulty_missed = faulty_missed * (-cost_production) # no revenue, just replacement cost
    expected_value = profit_good + profit_faulty_missed
    print(f"\nExpected value at cutoff {cutoff}: ${expected_value:.2f}")
```

```
>>> cost_production = 15
>>> sale_price = 25
>>> total_samples = len(df)
>>> actual_good = len(df[df['actual'] == 0])
>>> actual_faulty = len(df[df['actual'] == 1])
>>> for cutoff in cutoffs:
...     TP, TN, FP, FN = compute_confusion_matrix(df, cutoff)
...     false_negative_rate = FN / actual_faulty if actual_faulty > 0 else 0 # False negatives (faulty pieces missed)
...     profit_good = (actual_good / total_samples) * (sale_price - cost_production) # Profit from correctly selling good pieces
...     faulty_missed = false_negative_rate * actual_faulty
...     profit_faulty_missed = faulty_missed * (-cost_production) # no revenue, just replacement cost
...     expected_value = profit_good + profit_faulty_missed
...     print(f"\nExpected value at cutoff {cutoff}: ${expected_value:.2f}")
...

Expected value at cutoff 0.25: $8.50
Expected value at cutoff 0.75: -$6.50
```

Expected value at cutoff 0.25: \$8.50

Expected value at cutoff 0.75: -\$6.50

2) Bayes Rule and Conditional Probability

Individual Assignment 1

```
# 2. Bayes Rule and Conditional Probability -----
# Probability that a random person uses the drug
P_D1 = 0.0001
# Probability that a person does not use the drug
P_D0 = 1 - P_D1
# Drug test:
# Probability the test correctly identifies a user
P_T1_given_D1 = 0.97
# Probability the test correctly identifies a non-user
P_T0_given_D0 = 0.99
# 2a: Probability that a random person would test positive, P(T=1)
P_T1_given_D0 = 1 - P_T0_given_D0 # P(T=1 | D=0): false positive rate
P_T1 = (P_T1_given_D1 * P_D1) + (P_T1_given_D0 * P_D0)
print(f"Part 2a) - Probability that a person tests positive: {P_T1:.8f}")

# 2b: Calculate where most of these positive tests come from (true positives or false positives)
prob_true_positive = P_T1_given_D1 * P_D1
prob_false_positive = P_T1_given_D0 * P_D0
print(f"Part 2b) - True positives contribution: {prob_true_positive:.8f}")
print(f"Part 2b) - False positives contribution: {prob_false_positive:.8f}")
if prob_true_positive > prob_false_positive:
    print("Most positive tests are true positives.")
else:
    print("Most positive tests are false positives.")

# 2c: Probability that a random person who tests positive is a user, P(D=1|T=1)
P_D1_given_T1 = (prob_true_positive) / P_T1
print(f"Part 2c) - Probability that a person who tests positive is a user: {P_D1_given_T1:.8f}")

# 2d: Given this test to a random person and it came back positive, are they likely to be a drug user
print(f"Part 2d) - Given a positive test, the probability the person is a user: {P_D1_given_T1:.8f}")
```

```
>>> # Drug test:
>>> # Probability the test correctly identifies a user
>>>
>>> P_T1_given_D1 = 0.97
>>> # Probability the test correctly identifies a non-user
>>>
>>> P_T0_given_D0 = 0.99
>>> # 2a: Probability that a random person would test positive, P(T=1)
>>>
>>> P_T1_given_D0 = 1 - P_T0_given_D0 # P(T=1 | D=0): false positive rate
>>> P_T1 = (P_T1_given_D1 * P_D1) + (P_T1_given_D0 * P_D0)
>>> print(f"Part 2a) - Probability that a person tests positive: {P_T1:.8f}")
Part 2a) - Probability that a person tests positive: 0.01009600
>>> # 2b: Calculate where most of these positive tests come from (true positives or false positives)
>>>
>>> prob_true_positive = P_T1_given_D1 * P_D1
>>> prob_false_positive = P_T1_given_D0 * P_D0
>>> print(f"Part 2b) - True positives contribution: {prob_true_positive:.8f}")
Part 2b) - True positives contribution: 0.00009700
>>> print(f"Part 2b) - False positives contribution: {prob_false_positive:.8f}")
Part 2b) - False positives contribution: 0.00999900
>>> if prob_true_positive > prob_false_positive:
...     print("Most positive tests are true positives.")
... else:
...     print("Most positive tests are false positives.")
... # 2c: Probability that a random person who tests positive is a user, P(D=1|T=1)
...
Most positive tests are false positives.
>>> P_D1_given_T1 = (prob_true_positive) / P_T1
>>> print(f"Part 2c) - Probability that a person who tests positive is a user: {P_D1_given_T1:.8f}")
Part 2c) - Probability that a person who tests positive is a user: 0.00960777
>>> # 2d: Given this test to a random person and it came back positive, are they likely to be a drug user
>>>
>>> print(f"Part 2d) - Given a positive test, the probability the person is a user: {P_D1_given_T1:.8f}")
Part 2d) - Given a positive test, the probability the person is a user: 0.00960777
```

Individual Assignment 1

- a. The probability that a random person would test positive is 0.01009600 which is approximately 1.01%.
- b. Most positive tests are false positives which contribute about 0.00999900 (about 1%). The true positives contribution is 0.00009700 (about 0.01%)
- c. The probability that a person who tests positive is a drug user is approximately 0.00960777, or about 0.96%.
- d. If I have given this test to a random person and it came back positive; the probability the person is likely to be a drug user is 0.00960777 (about 1%). So they are very unlikely to be a drug user.

3) Multiple Linear Regression

- a) If I were to create a model using Multiple Linear Regression, my target variable would be *Age*, which is the age of the fossil in millions of years. My predicting variables would be *Brain size*, *Body size*, and *Age group*.
- b) The data includes details about different fossils, like how old they are in millions of years, where they were found, their taxonomy, brain size, body size, their age group, and the methods used to date them. The dataset has quite a few columns because categorical info like location, taxonomy, and dating methods are turned into dummy variables. The age of the fossils varies across a range, and from looking at some initial patterns, it seems like brain size and age group may have a pretty strong link to the age. The sizes are continuous variables, while age group is categorical, so it's represented with dummy variables. Overall, these variables seem good for predicting the age, but it's important to check the actual relationships through correlation or scatterplots to be sure.
- c) Variables like brain size and age group are related to the age of the fossils, which means a multiple linear regression model could predict the age well. The initial signs are that these variables have a strong connection to age, so they could be useful predictors. However, to be confident and to see how strong the relationship is, I would do a detailed analysis, like looking at correlation matrices or scatterplots.
- d) a.

Individual Assignment 1

```
# 3. Multiple Linear Regression -----
# Load the dataset
file_path = r"C:\Users\bhuva\Downloads\DATA 611\Assignment\Paleontology.csv"
df = pd.read_csv(file_path)

# d) Testing the Multiple Linear Regression Model [X = 'Brain size', 'Body size', 'Age group' and Y = 'Age (Myr)']
X = df[['Brain size', 'Body size', 'Age group']]
y = df['Age (Myr)']

# 3d. a) Partition data into training (80%) and testing (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Sample of training data
# Combine X_train with y_train
train_combined = X_train.copy()
train_combined['Age (Myr)'] = y_train
print("\nSample of combined training data (X_train + y_train):")
print(train_combined.head())

# Sample of Testing data
# Combine X_test with y_test
test_combined = X_test.copy()
test_combined['Age (Myr)'] = y_test
print("\nSample of combined testing data (X_test + y_test):")
print(test_combined.head())
```

```
>>> file_path = r"C:\Users\bhuva\Downloads\DATA 611\Assignment\Paleontology.csv"
>>> df = pd.read_csv(file_path)
>>> # d) Testing the Multiple Linear Regression Model [X = 'Brain size', 'Body size', 'Age group' and Y = 'Age (Myr)']
>>>
>>> X = df[['Brain size', 'Body size', 'Age group']]
>>> y = df['Age (Myr)']
>>> # 3d. a) Partition data into training (80%) and testing (20%)
>>>
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.2, random_state=42
... )
>>> # Sample of training data
>>> # Combine X_train with y_train
>>>
>>> train_combined = X_train.copy()
>>> train_combined['Age (Myr)'] = y_train
>>> print("\nSample of combined training data (X_train + y_train):")

Sample of combined training data (X_train + y_train):
>>> print(train_combined.head())
   Brain size  Body size  Age group  Age (Myr)
34    1490.0     81.7         1     0.0515
30    1261.0     82.6         1     0.0400
8     1700.0     67.4         1     0.0180
17    1880.0     82.3         1     0.0290
6     1484.0     66.9         1     0.0150
>>> # Sample of Testing data
>>> # Combine X_test with y_test
>>>
>>> test_combined = X_test.copy()
>>> test_combined['Age (Myr)'] = y_test
>>> print("\nSample of combined testing data (X_test + y_test):")

Sample of combined testing data (X_test + y_test):
>>> print(test_combined.head())
   Brain size  Body size  Age group  Age (Myr)
0     1520.0     67.6         1     0.011
5     1597.0     58.8         1     0.014
33    1550.0     71.2         1     0.048
13    1475.0     74.9         1     0.023
19    1590.0     71.2         1     0.031
```

Individual Assignment 1

Sample of training data:

```
Sample of combined training data (X_train + y_train):
>>> print(train_combined.head())
   Brain size  Body size  Age group  Age (Myr)
34      1490.0      81.7         1    0.0515
30      1261.0      82.6         1    0.0400
8       1700.0      67.4         1    0.0180
17      1880.0      82.3         1    0.0290
6       1484.0      66.9         1    0.0150
```

Sample of testing data:

```
Sample of combined testing data (X_test + y_test):
>>> print(test_combined.head())
   Brain size  Body size  Age group  Age (Myr)
0       1520.0      67.6         1    0.011
5       1597.0      58.8         1    0.014
33      1550.0      71.2         1    0.048
13      1475.0      74.9         1    0.023
19      1590.0      71.2         1    0.031
```

d) b.

```
# 3d. b) Fit the multiple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)
coefficients = pd.Series(model.coef_, index=X.columns)
intercept = model.intercept_

equation = f"Age (Myr) = {intercept:.4f}"
for name, coef in coefficients.items():
    equation += f" + ({coef:.4f} * {name})"
print("\nModel Equation:\n", equation)
```

```
>>> model = LinearRegression()
>>> model.fit(X_train, y_train)
LinearRegression()
>>> coefficients = pd.Series(model.coef_, index=X.columns)
>>> intercept = model.intercept_
>>> equation = f"Age (Myr) = {intercept:.4f}"
>>> for name, coef in coefficients.items():
...     equation += f" + ({coef:.4f} * {name})"
...
>>> print("\nModel Equation:\n", equation)

Model Equation:
Age (Myr) = -0.0472 + (0.0000 * Brain size) + (-0.0005 * Body size) + (0.1020 * Age group)
```

The model equation:

$$\text{Age (Myr)} = -0.0472 + (0.0000 * \text{Brain size}) + (-0.0005 * \text{Body size}) + (0.1020 * \text{Age group})$$

Simplified equation:

$$\text{Age (Myr)} = -0.0472 + (-0.0005 * \text{Body size}) + (0.1020 * \text{Age group})$$

Individual Assignment 1

d) c.

```
# 3d. c) Evaluate the models performance
y_pred_test = model.predict(X_test)
r2 = r2_score(y_test, y_pred_test)
mae = mean_absolute_error(y_test, y_pred_test)
mse = mean_squared_error(y_test, y_pred_test)
rmse = np.sqrt(mse)

print("\nModel performance on test data:")
print(f"R^2 score: {r2:.3f}")
print(f"MAE: {mae:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
```

```
>>> y_pred_test = model.predict(X_test)
>>> r2 = r2_score(y_test, y_pred_test)
>>> mae = mean_absolute_error(y_test, y_pred_test)
>>> mse = mean_squared_error(y_test, y_pred_test)
>>> rmse = np.sqrt(mse)
>>> print("\nModel performance on test data:")

Model performance on test data:
>>> print(f"R^2 score: {r2:.3f}")
R^2 score: 0.987
>>> print(f"MAE: {mae:.4f}")
MAE: 0.0201
>>> print(f"MSE: {mse:.4f}")
MSE: 0.0006
>>> print(f"RMSE: {rmse:.4f}")
RMSE: 0.0253
```

Model's score (R2): 0.987

Mean Absolute Error (MAE): 0.0201

Mean Squared Error (MSE): 0.0006

Root Mean Square Error (RMSE): 0.0253

d) d.

The model explains 98.7% of the variation in the data ($R^2 = 0.987$), which is a really good fit. The predictions are off by about 20,100 years (MAE = 0.0201 million years). The model also has a very small error (MSE = 0.0006 and RMSE = 0.0253 million years), meaning the predictions are very close to the actual ages. Thus, the model predicts the test data very accurately, so it's pretty reliable for estimating fossil ages using brain size, body size, and age group. It could help in estimating ages of new fossils quickly and accurately.

```
# 3d. d) Conclusions
print(f"\nConclusion: The model explains approximately {r2*100:.2f}% of the variance in age.")
print("The MAE and RMSE shows the average prediction errors, explaining how well the model predicts age.")
```

```
Conclusion: The model explains approximately 98.71% of the variance in age.
>>> print("The MAE and RMSE shows the average prediction errors, explaining how well the model predicts age.")
The MAE and RMSE shows the average prediction errors, explaining how well the model predicts age.
```

Individual Assignment 1

d) e.

```
# 3d. e) Training error comparison
y_pred_train = model.predict(X_train)
train_mae = mean_absolute_error(y_train, y_pred_train)
train_mse = mean_squared_error(y_train, y_pred_train)
train_rmse = np.sqrt(train_mse)
print("\nTraining errors:")
print(f"MAE: {train_mae:.4f}")
print(f"MSE: {train_mse:.4f}")
print(f"RMSE: {train_rmse:.4f}")

>>> y_pred_train = model.predict(X_train)
>>> train_mae = mean_absolute_error(y_train, y_pred_train)
>>> train_mse = mean_squared_error(y_train, y_pred_train)
>>> train_rmse = np.sqrt(train_mse)
>>> print("\nTraining errors:")

Training errors:
>>> print(f"MAE: {train_mae:.4f}")
MAE: 0.0171
>>> print(f"MSE: {train_mse:.4f}")
MSE: 0.0005
>>> print(f"RMSE: {train_rmse:.4f}")
RMSE: 0.0229
```

Training MAE: 0.0171

MSE: 0.0005

RMSE: 0.0229

There are only minor differences between the MAEs (0.0171 vs. 0.0201), MSEs (0.0005 vs. 0.0006), and RMSEs (0.0229 vs. 0.0253) of the training and test datasets. Thus, the model performs similarly for both dataset types, and has a high likelihood of performing equally well on future data. The closeness of training and test data errors suggests that the model's fossil age estimates have a high level of reliability and that there is no overfitting problem affecting the model's ability to generalize.

d) f.

```
# 3d. f) Predict age of new specimen
# Given data:
brain_size_new = 1453
body_size_new = 68
age_group_new = 1

# Prepare input with training feature names
new_specimen = pd.DataFrame({
    'Brain size': [brain_size_new],
    'Body size': [body_size_new],
    'Age group': [age_group_new]
})

# Predict age using the trained model
predicted_age = model.predict(new_specimen)[0]
print(f"\nPredicted age for new specimen:")
print(f"Predicted Age (Million years): {predicted_age:.2f}")
```

Individual Assignment 1

```
>>> brain_size_new = 1453
>>> body_size_new = 68
>>> age_group_new = 1
>>> # Prepare input with training feature names
>>>
>>> new_specimen = pd.DataFrame({
...     'Brain size': [brain_size_new],
...     'Body size': [body_size_new],
...     'Age group': [age_group_new]
... })
>>> # Predict age using the trained model
>>>
>>> predicted_age = model.predict(new_specimen)[0]
>>> print(f"\nPredicted age for new specimen:")

Predicted age for new specimen:
>>> print(f"Predicted Age (Million years): {predicted_age:.2f}")
Predicted Age (Million years): 0.04
```

Based on the model, the new specimen's estimated age is about 0.04 million years.

4.

- a) The maximum number of leaves in a binary tree of depth L is 2^L .
- b) The maximum number of internal nodes (excluding leaves) is $2^L - 1$.
- c) For cross-validation, a larger k results in more reliable results, which is an advantage. A disadvantage is that a larger k requires increased computation time and power.
- d) To create a classifier that is invariant to small shifts in audio, use features, such as MFCCs.
- e) Squared error gives greater weight to larger errors and therefore will increase the ease of optimization.
- f) Increasing λ decreases the number of features available by moving the associated weight towards 0, while balancing between the model's simplicity and fit.

5. a)

```
# 5. K-Nearest Neighbors -----
# a) Define predictor variables and target variable
predictor_cols = ['Age (Myr)', 'Brain size', 'Body size']
target_col = 'Taxonomy'

# Snapshot for the first 10 observations for the predictor variables and target variable
snapshot = df[predictor_cols + [target_col]].head(10)
print("First 10 observations of target and predictor variables:\n")
print(snapshot)
```

Individual Assignment 1

```
>>> predictor_cols = ['Age (Myr)', 'Brain size', 'Body size']
>>> target_col = 'Taxonomy'
>>> # Snapshot for the first 10 observations for the predictor variables and target variable
>>>
>>> snapshot = df[predictor_cols + [target_col]].head(10)
>>> print("First 10 observations of target and predictor variables:\n")
First 10 observations of target and predictor variables:

>>> print(snapshot)
   Age (Myr)  Brain size  Body size  Taxonomy
0    0.011    1520.0    67.6  Pleistocene Homo sapiens
1    0.011    1661.0    66.5  Pleistocene Homo sapiens
2    0.012    1424.0    73.3  Pleistocene Homo sapiens
3    0.014    1492.0    83.8  Pleistocene Homo sapiens
4    0.014    1330.0    75.1  Pleistocene Homo sapiens
5    0.014    1597.0    58.8  Pleistocene Homo sapiens
6    0.015    1484.0    66.9  Pleistocene Homo sapiens
7    0.018    1434.0    59.4  Pleistocene Homo sapiens
8    0.018    1700.0    67.4  Pleistocene Homo sapiens
9    0.019    1354.0    59.2  Pleistocene Homo sapiens
```

b)

```
# b) Partition into 90% train and 10% test
# Check if these columns exist
for col in predictor_cols + [target_col]:
    if col not in df.columns:
        raise ValueError(f"Column '{col}' not found in dataset.")

X = df[predictor_cols]
y = df[target_col]

# Encode target variable
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Ensure predictor columns are numeric
X = X.select_dtypes(include=[np.number])

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.1, random_state=42
)

# Snapshot of the training data
print("Training data snapshot:\n")
print(pd.concat([X_train, pd.Series(y_train, name='Taxonomy')], axis=1).head())

# Snapshot of the test data
print("\nTest data snapshot:\n")
print(pd.concat([X_test, pd.Series(y_test, name='Taxonomy')], axis=1).head())
```

Individual Assignment 1

```
>>> for col in predictor_cols + [target_col]:
...     if col not in df.columns:
...         raise ValueError(f"Column '{col}' not found in dataset.")
...
>>> X = df[predictor_cols]
>>> y = df[target_col]
>>> # Encode target variable
>>>
>>> le = LabelEncoder()
>>> y_encoded = le.fit_transform(y)
>>> # Ensure predictor columns are numeric
>>>
>>> X = X.select_dtypes(include=[np.number])
>>> # Split data
>>>
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y_encoded, test_size=0.1, random_state=42
... )
>>> # Snapshot of the training data
>>>
>>> print("Training data snapshot:\n")
Training data snapshot:

>>> print(pd.concat([X_train, pd.Series(y_train, name='Taxonomy')], axis=1).head())
   Age (Myr)  Brain size  Body size  Taxonomy
36    0.095    1525.0    46.7         1.0
26    0.035    1606.0    65.6         1.0
44    0.195    1491.0    54.1         2.0
12    0.021    1090.0    47.4         2.0
54    0.770    1030.0    54.8         NaN
```

```
>>> # Snapshot of the test data
>>>
>>> print("\nTest data snapshot:\n")

Test data snapshot:

>>> print(pd.concat([X_test, pd.Series(y_test, name='Taxonomy')], axis=1).head())
   Age (Myr)  Brain size  Body size  Taxonomy
0    0.011    1520.0    67.6         2.0
5    0.014    1597.0    58.8         0.0
33   0.048    1550.0    71.2         NaN
13   0.023    1475.0    74.9         NaN
19   0.031    1590.0    71.2         NaN
```

c)

```
# c) Create a k-NN classifier with k=3 and fit it
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("\nPredicted labels for test data:\n")
print(le.inverse_transform(y_pred))
```

```
>>> knn = KNeighborsClassifier(n_neighbors=3)
>>> knn.fit(X_train, y_train)
KNeighborsClassifier(n_neighbors=3)
>>> y_pred = knn.predict(X_test)
>>> print("\nPredicted labels for test data:\n")

Predicted labels for test data:

>>> print(le.inverse_transform(y_pred))
['Pleistocene Homo sapiens' 'Pleistocene Homo sapiens'
 'Pleistocene Homo sapiens' 'Pleistocene Homo sapiens'
 'Pleistocene Homo sapiens' 'Mid-Pleistocene Homo']
```

Individual Assignment 1

A k-NN model with $k = 3$ was trained on the training data. The model classified five of them as 'Pleistocene Homo sapiens' and one as 'Mid-Pleistocene Homo'.

d)

```
# d) Show confusion matrix and accuracy
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)
print(f"\nAccuracy: {accuracy*100:.2f}%")

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

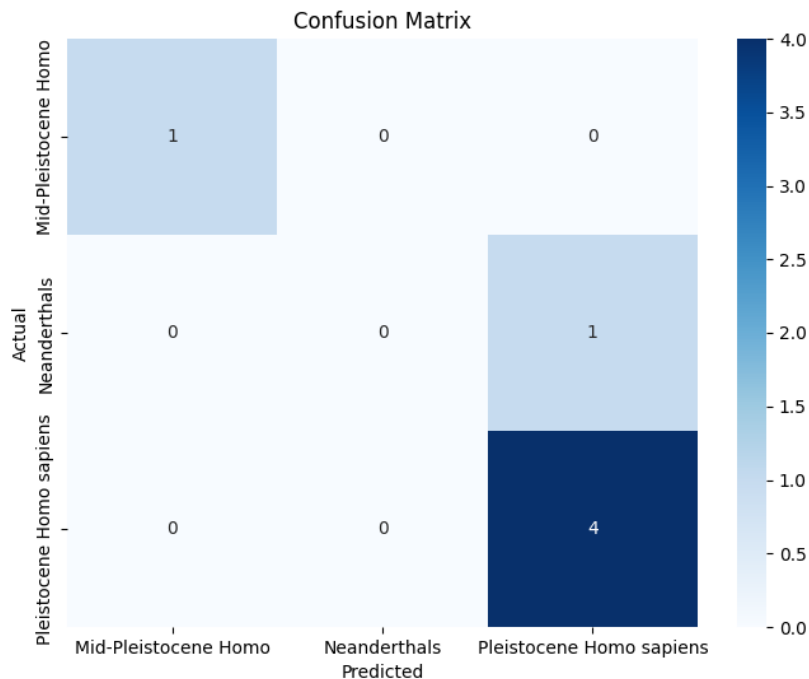
```
>>> cm = confusion_matrix(y_test, y_pred)
>>> accuracy = accuracy_score(y_test, y_pred)
>>> print("\nConfusion Matrix:\n", cm)

Confusion Matrix:
[[1 0 0]
 [0 0 1]
 [0 0 4]]
>>> print(f"\nAccuracy: {accuracy*100:.2f}%")

Accuracy: 83.33%
```

```
>>> plt.figure(figsize=(8,6))
<Figure size 800x600 with 0 Axes>
>>> sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
...             xticklabels=le.classes_, yticklabels=le.classes_)
<Axes: >
>>>
>>> plt.xlabel('Predicted')
Text(0.5, 36.72222222222221, 'Predicted')
>>> plt.ylabel('Actual')
Text(70.72222222222222, 0.5, 'Actual')
>>> plt.title('Confusion Matrix')
Text(0.5, 1.0, 'Confusion Matrix')
>>> plt.show()
```

Individual Assignment 1



The classification model has properly classified 5 out of 6 tested instances, resulting in an accuracy of around 83.33%. The confusion matrix shows that the majority of all classifications were classified correctly with only 1 instance being misclassified (which was a Neanderthal misclassified as a Pleistocene Homo sapien). Therefore, the classification model shows a good capacity to predict accurately and appropriately differentiate among 3 taxonomies modelled, demonstrating high-degree of predictive accuracy.

e)

```
# e) Is k=3 the 'best' k?
k_values = list(range(1, 20, 2))
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    accuracies.append(accuracy_score(y_test, pred))

# Plot accuracy vs k
plt.figure(figsize=(8,6))
plt.plot(k_values, accuracies, marker='o')
plt.xlabel('k')
plt.ylabel('Test Accuracy')
plt.title('KNN Accuracy for different k values')
plt.show()

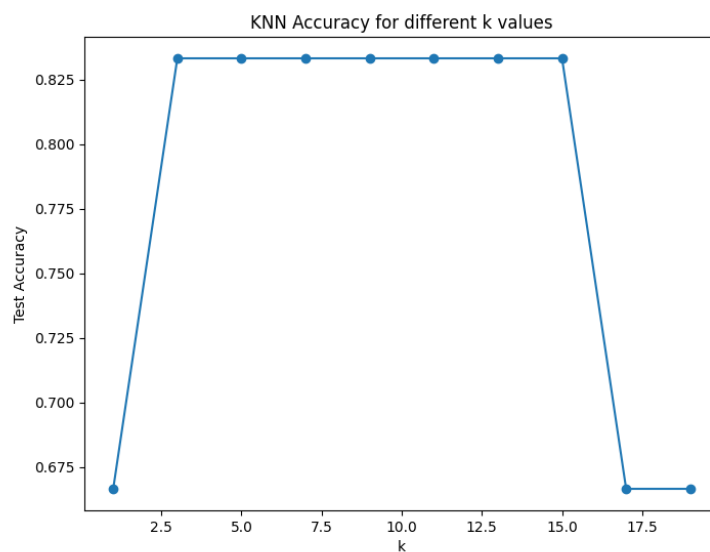
# Find the best k
best_k = k_values[np.argmax(accuracies)]
print(f"\nThe best k based on test accuracy is: {best_k}")
```

Individual Assignment 1

```
>>> k_values = list(range(1, 20, 2))
>>> accuracies = []
>>> for k in k_values:
...     model = KNeighborsClassifier(n_neighbors=k)
...     model.fit(X_train, y_train)
...     pred = model.predict(X_test)
...     accuracies.append(accuracy_score(y_test, pred))
... # Plot accuracy vs k
...
KNeighborsClassifier(n_neighbors=1)
KNeighborsClassifier(n_neighbors=3)
KNeighborsClassifier()
KNeighborsClassifier(n_neighbors=7)
KNeighborsClassifier(n_neighbors=9)
KNeighborsClassifier(n_neighbors=11)
KNeighborsClassifier(n_neighbors=13)
KNeighborsClassifier(n_neighbors=15)
KNeighborsClassifier(n_neighbors=17)
KNeighborsClassifier(n_neighbors=19)
>>> plt.figure(figsize=(8,6))
<Figure size 800x600 with 0 Axes>
>>> plt.plot(k_values, accuracies, marker='o')
[<matplotlib.lines.Line2D object at 0x0000018942A7AE90>]
>>> plt.xlabel('k')
<Figure size 800x600 with 0 Axes>
>>> plt.plot(k_values, accuracies, marker='o')
[<matplotlib.lines.Line2D object at 0x0000018942A7AE90>]
>>> plt.xlabel('k')
>>> plt.plot(k_values, accuracies, marker='o')
[<matplotlib.lines.Line2D object at 0x0000018942A7AE90>]
>>> plt.xlabel('k')
[<matplotlib.lines.Line2D object at 0x0000018942A7AE90>]
>>> plt.xlabel('k')
>>> plt.xlabel('k')
Text(0.5, 0, 'k')
```

```
>>> plt.ylabel('Test Accuracy')
Text(0, 0.5, 'Test Accuracy')
>>> plt.title('KNN Accuracy for different k values')
Text(0.5, 1.0, 'KNN Accuracy for different k values')
>>> plt.show()
>>> # Find the best k
>>>
>>> best_k = k_values[np.argmax(accuracies)]
>>> print(f"\nThe best k based on test accuracy is: {best_k}")

The best k based on test accuracy is: 3
```



Individual Assignment 1

The accuracy plot shows that the highest test accuracy (~83%) occurs when $k=3$. The k values around 3 produce a model that maintains a high level of test accuracy by balancing bias and variance. Thus, $k=3$ is likely the most suitable "k" value for these data.

f)

```
# f) Classify the taxonomy
# Create DataFrame with new specimen data
new_specimen_df = pd.DataFrame(
    [[0.33, 1453, 68]],
    columns=predictor_cols # ['Age (Myr)', 'Brain size', 'Body size']
)

# Predict
predicted_taxonomy = le.inverse_transform(knn.predict(new_specimen_df))
print(f"\nThe predicted taxonomy for the new specimen is: {predicted_taxonomy[0]}")
```

```
>>> new_specimen_df = pd.DataFrame(
...     [[0.33, 1453, 68]],
...     columns=predictor_cols # ['Age (Myr)', 'Brain size', 'Body size']
... )
>>> # Predict
>>>
>>> predicted_taxonomy = le.inverse_transform(knn.predict(new_specimen_df))
>>> print(f"\nThe predicted taxonomy for the new specimen is: {predicted_taxonomy[0]}")

The predicted taxonomy for the new specimen is: Pleistocene Homo sapiens
```

The predicted taxonomy for the new specimen is: Pleistocene Homo sapiens